GOBIERNO DE MÉXICO | EDUCACIÓN SECRETARÍA DE EDUCACIÓN PÚBLICA | TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO SUPERIOR DE XALAPA

MATERIA: Métodos Numéricos

PROFESOR: Víctor Hugo Vázquez Herrera

CARRERA: Ingeniería en Sistemas Computacionales.

SEMESTRE: 3° Sabatino

GRUPO: "B"

Alumna: Fatima Itzel Martínez Salamanca

Matrícula: 257o03058

Tarea 1 unidad 5

INSTITUTO TECNOLÓGICO SUPERIOR DE XALAPA

Xalapa ver. 9 de Noviembre del 2026

1. Entrega de Código y Repositorio GitHub

Como parte de las actividades prácticas evaluables correspondientes a la Unidad 5 de la asignatura Métodos Numéricos, se ha preparado y publicado la implementación completa de los algoritmos de interpolación polinómica en un repositorio público individual dentro de la plataforma GitHub.

1.1 Enlace Oficial al Repositorio

URL del Proyecto:
https://github.com/FatimaItzelMartinezSalamanca10/Metodos_Numericos/tree/main/interpolaci%C3%B3n

1.2 Estructura del Archivo README.md

El repositorio cuenta obligatoriamente con un archivo README.md que sirve como guía de usuario para el entorno de desarrollo y ejecución rápida. Su contenido incluye:

- **Estructura del Paquete:** Organización de las clases en el paquete explícito `interpolación` (`InterpolacionLagrange.java`, `InterpolacionNewton.java` y `Main.java`).
- **Instrucciones de Compilación y Ejecución:** Comandos para compilar desde la terminal integrada o ejecutar directamente mediante la clase principal `Main.java`.
- **Entrada y Salida Esperada:** Descripción detallada de los arreglos de entrada (nodos x e y), el punto intermedio a interpolar (xEval = 3.0) y el formato estructurado de los resultados producidos.

2. Introducción

En nuestro día a día con la ingeniería y el análisis numérico, nos topamos muy seguido con tablas de datos que salieron de experimentos, mediciones o muestreos reales, pero de las cuales no conocemos la fórmula matemática que las originó. Cuando necesitamos calcular un valor que queda en medio de esos puntos y que no medimos directamente, es ahí donde entra la interpolación polinómica. Lo que busca este proceso es armar un polinomio P(x) que pase exactamente por todos nuestros puntos o nodos conocidos (x_0, y_0), (x_1, y_1), ..., (x_n, y_n).

Para llegar a ese polinomio hay varios caminos, siendo dos de los más importantes el método de Lagrange y el de Diferencias Divididas de Newton. La teoría nos respalda con el Teorema de Unicidad, el cual nos dice que, sin importar el método que elijas, para un conjunto de n + 1 puntos distintos siempre va a existir un único polinomio de grado menor o igual a n que los cruza a todos. Sin embargo, por dentro funcionan muy diferente y su rendimiento al programarlos cambia bastante. Por eso, en este reporte práctico implementamos ambos métodos en Java para ponerlos a prueba, comparar cómo resuelven las operaciones y analizar cuál resulta más eficiente cuando trabajamos con más datos.

2.1 Fundamentos de la Interpolación Polinomial

En numerosas aplicaciones computacionales y numéricas, la función matemática subyacente f(x) es desconocida o resulta extremadamente compleja para evaluarse directamente. La interpolación permite reemplazar dicha función por un polinomio P_n(x) de grado menor o igual a n que aproxima con alta fidelidad el comportamiento de los datos.

2.2 Polinomios de Lagrange

El método desarrollado por Joseph-Louis Lagrange expresa el polinomio interpolador único mediante una combinación lineal de las ordenadas y_i ponderadas por polinomios base $L_i(x)$:

$$P(x) = \sum [\ y_i \cdot L_i(x)\ ] \quad \text{para } i = 0, 1, \ldots, n$$

donde la función base polinómica $L_i(x)$ se define mediante la productoria:

$$L_i(x) = \prod [\ (x - x_j) / (x_i - x_j)\ ] \quad \text{para } j \neq i$$

La propiedad clave del polinomio base $L_i(x)$ es la delta de Kronecker: toma el valor 1 cuando se evalúa en el nodo x_i (es decir, $L_i(x_i) = 1$) y vale 0 cuando se evalúa en cualquier otro nodo x_j para $j \neq i$.

2.3 Método de Diferencias Divididas de Newton

El método de Sir Isaac Newton ofrece una formulación alternativa basada en la construcción progresiva del polinomio interpolador a través de diferencias divididas. El polinomio adopta la forma:

$$P(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \ldots + a_n(x - x_0)(x - x_1)\ldots(x - x_{n-1})$$

donde los coeficientes a_k equivalen a las diferencias divididas de orden k:

$$a_0 = f[x_0] = y_0$$
$$a_1 = f[x_0, x_1] = (y_1 - y_0) / (x_1 - x_0)$$
$$a_2 = f[x_0, x_1, x_2] = (f[x_1, x_2] - f[x_0, x_1]) / (x_2 - x_0)$$

2.4 Cuadro Comparativo Matemático

Aspecto	Método de Lagrange	Diferencias Divididas de Newton
Estructura	Suma de productoras base $L_i(x)$	Forma polinómica escalonada
Adición de nodos	Recálculo completo de todas las bases $L_i(x)$	Cálculo incremental de una nueva columna/fila
Evaluación	Evaluación directa $O(n^2)$	Construcción $O(n^2)$, evaluación $O(n)$

3. Desarrollo del Código (InterpolacionLagrange.java)

En la plantilla de código proporcionada para la práctica, la clase `InterpolacionLagrange.java` presentaba una línea incompleta dentro del ciclo interno encargada de computar la productoria del polinomio base.

3.1 Corrección Realizada en la Clase

A continuación se muestra el bloque corregido donde se implementó la fórmula exacta del producto base $L_i(x)$:

```
package interpolación;

public class InterpolacionLagrange {

    private static void validarDatos(double[] x, double[] y) {
        // Revisar que los arreglos existan y tengan datos
        if (x == null || y == null || x.length == 0 || y.length == 0) {
            throw new IllegalArgumentException("Los arreglos no pueden ser nulos ni
estar vacios.");
        }
        // Checar que tengan la misma cantidad de elementos
        if (x.length != y.length) {
            throw new IllegalArgumentException("Los arreglos x y y deben tener el
mismo tamano.");
        }
        // Si hay una x repetida, el denominador se hace cero y truena
        for (int i = 0; i < x.length; i++) {
            for (int j = i + 1; j < x.length; j++) {
                if (x[i] == x[j]) {
                    throw new IllegalArgumentException("Nodos de x repetidos: " +
x[i]);
                }
            }
        }
    }

    public static double interpolacionLagrange(double[] x, double[] y, double xEval) {
        validarDatos(x, y);
        double resultado = 0.0;
        int n = x.length;

        for (int i = 0; i < n; i++) {
            // Empezamos guardando el valor de y en esta posicion
            double termino = y[i];
            for (int j = 0; j < n; j++) {
                // Hacemos las multiplicaciones ignorando el nodo i
                if (i != j) {
                    termino *= (xEval - x[j]) / (x[i] - x[j]);
                }
            }
            // Vamos sumando lo que dio esta parte al resultado
            resultado += termino;
        }
        return resultado;
    }
}
```

3.2 Justificación Técnica

Justificación: La variable `termino` se inicializa con el valor y[i]. Dentro del ciclo `for (int j = 0; j < n; j++)`, la condición `if (i != j)` evita la división por cero cuando i coincide con j. En cada paso válido, el término se multiplica acumulativamente por el cociente `(xEval - x[j]) / (x[i] - x[j])`. Al finalizar el ciclo interno, `termino` almacena el valor exacto de $y_i \cdot L_i(xEval)$, el cual se suma directamente a la variable `resultado`.

3.3 Clases Auxiliares (InterpolacionNewton.java y Main.java)

Para completar la estructura modular del proyecto en Java, se integraron las clases correspondientes al método de Newton y la clase principal:

```java
package interpolación;

public class InterpolacionNewton
{

    /**
     * Calcula la tabla de diferencias divididas para los puntos dados.
     *
     * @param x Arreglo con los valores de x (nodos)
     * @param y Arreglo con los valores de y (f(x))
     * @return Matriz con la tabla de diferencias divididas
     */
    public static double[][] calcularDiferenciasDivididas(double[] x, double[] y) {
        int n = x.length;
        double[][] tabla = new double[n][n];

        // La primera columna contiene los valores de y (diferencias de orden 0)
        for (int i = 0; i < n; i++) {
            tabla[i][0] = y[i];
        }

        // Cálculo de las diferencias divididas de orden superior
        for (int j = 1; j < n; j++) {
            for (int i = 0; i < n - j; i++) {
                tabla[i][j] = (tabla[i + 1][j - 1] - tabla[i][j - 1]) / (x[i + j] - x[i]);
            }
        }
        return tabla;
    }




    /**
     * Evalúa el polinomio interpolador de Newton en un punto xEval dado.
     *
     * @param x     Arreglo con los nodos de x
     * @param tabla Matriz de diferencias divididas
     * @param xEval El punto x donde queremos interpolar
     * @return El valor interpolado y = P(xEval)
     */
    public static double evaluarPolinomio(double[] x, double[][] tabla, double xEval) {
        int n = x.length;
        double resultado = tabla[0][0];
        double productoTerminos = 1.0;

        // Sumamos a1*(x - x0) + a2*(x - x0)(x - x1) + ...
        for (int i = 1; i < n; i++) {
            productoTerminos *= (xEval - x[i - 1]);
            resultado += tabla[0][i] * productoTerminos;
```

```
        }

        return resultado;
    }




    /**
     * Imprime en consola la tabla de diferencias divididas de forma estructurada.
     */
    public static void imprimirTabla(double[] x, double[][] tabla) {
        int n = x.length;
        System.out.println("\n--- TABLA DE DIFERENCIAS DIVIDIDAS ---");
        System.out.print("x_i\tf[x_i]\t");
        for (int i = 1; i < n; i++) {
            System.out.print("Orden " + i + "\t");
        }
        System.out.println("\n-------------------------------------------------------");

        for (int i = 0; i < n; i++) {
            System.out.printf("%.2f\t", x[i]);
            for (int j = 0; j < n - i; j++) {
                System.out.printf("%.4f\t", tabla[i][j]);
            }
            System.out.println();
        }
    }
}
```

Main.Java

```
package interpolación;

public class Main {

    public static void main(String[] args) {
        //Nodos tomados de f(x)= x² + 1
        double[] x = {1.0, 2.0, 4.0};
        double[] y = {2.0, 5.0, 17.0};

        // Punto intermedio a interpolar
        double xEval = 3.0;

        // 1. Calcular tabla de diferencias divididas llamando a la clase de
interpolación
        double[][] tablaDiferencias =
InterpolacionNewton.calcularDiferenciasDivididas(x, y);

        // 2. Imprimir la tabla
        InterpolacionNewton.imprimirTabla(x, tablaDiferencias);

        // 3. Mostrar los coeficientes (Diagonal superior)
        System.out.println("\n--- COEFICIENTES DEL POLINOMIO (a_k) ---");
        for (int k = 0; k < x.length; k++) {
            System.out.printf("a_%d = %.4f\n", k, tablaDiferencias[0][k]);
        }

        // 4. Evaluar en x = 3
        double yInterpolado = InterpolacionNewton.evaluarPolinomio(x,
```

```
tablaDiferencias, xEval);

        System.out.println("\n--- RESULTADO DE LA INTERPOLACIÓN ---");
        System.out.printf("El valor estimado en x = %.2f es P(%.2f) =
%.4f\n", xEval, xEval, yInterpolado);
    }
}
```

4. Pruebas Numéricas y Resultados en Consola

Se probó la validez computacional de las implementaciones utilizando los nodos tomados de la función cuadrática $f(x) = x^2 + 1$ para comprobar la coincidencia de los métodos:

- **Nodos de Entrada:** x = [1.0, 2.0, 4.0], y = [2.0, 5.0, 17.0]
- **Punto de Evaluación:** xEval = 3.0
- **Valor Real Teórico de Referencia:** $f(3.0) = (3.0)^2 + 1 = 10.0$

4.1 Tabla de Diferencias Divididas Impresa por Consola

x_i	f[x_i] (Orden 0)	Orden 1	Orden 2 (a_k)
1.00	2.0000	3.0000	1.0000
2.00	5.0000	6.0000	—
4.00	17.0000	—	—

4.2 Salida Textual Registrada en Consola (IntelliJ IDEA)

```
--- TABLA DE DIFERENCIAS DIVIDIDAS ---
x_i       f[x_i]     Orden 1  Orden 2
-----------------------------------------------------
1.00      2.0000   3.0000   1.0000
2.00      5.0000   6.0000
4.00      17.0000

--- COEFICIENTES DEL POLINOMIO (a_k) ---
a_0 = 2.0000
a_1 = 3.0000
a_2 = 1.0000

--- RESULTADO DE LA INTERPOLACIÓN ---
El valor estimado en x = 3.00 es P(3.00) = 10.0000

Process finished with exit code 0
```

5. Análisis Comparativo y Preguntas de Discusión

1. ¿Ambos métodos entregan exactamente el mismo resultado para el punto evaluado?

Sí. Ambos métodos entregan exactamente el valor P(3.0) = 10.0000. Esto valida numéricamente el Teorema de Unicidad del Polinomio Interpolador: para un conjunto de (n + 1) nodos distintos, existe un único polinomio de grado menor o igual a n que satisface la condición de interpolación. Lagrange y Newton son simplemente dos formulaciones algebraicas distintas de un mismo objeto matemático.

2. Si se añade un nuevo punto al conjunto inicial, ¿cuál método exige menor esfuerzo computacional?

El método de Diferencias Divididas de Newton exige significativamente menor esfuerzo. Al incorporar un nodo adicional (x_{n+1}, y_{n+1}), Newton conserva intactos todos los coeficientes a_k previamente calculados y solo requiere añadir una nueva renglón/columna a la matriz. En cambio, en Lagrange se deben recalcular por completo todos los polinomios base $L_i(x)$ desde el principio.

3. ¿Qué complejidad algorítmica O(n) presenta la evaluación de cada algoritmo?

• Lagrange: Complejidad $O(n^2)$ para evaluar un punto individual, requiriendo n^2 multiplicaciones y divisiones.
• Newton: Construir la tabla de diferencias divididas toma $O(n^2)$, pero una vez almacenados los coeficientes, la evaluación del polinomio en cualquier punto x mediante la Regla de Horner toma únicamente O(n) operaciones.

6. Conclusiones

La realización de esta práctica me permitió aplicar de forma directa dos de los métodos de interpolación polinómica más representativos del análisis numérico, llevando los conceptos matemáticos a una implementación funcional en Java. Al trabajar con los datos de prueba y comparar las salidas de ambos algoritmos, pude corroborar que tanto el método de Polinomios de Lagrange como el de Diferencias Divididas de Newton conducen exactamente al mismo valor estimado para el punto evaluado. Este resultado no solo valida la corrección del código desarrollado, sino que pone de manifiesto la unicidad del polinomio interpolador: sin importar el procedimiento algebraico que utilicemos para construirlo, existe un solo polinomio de grado menor o igual a n que pasa por el conjunto de puntos dados.

Más allá del resultado numérico, el valor principal de este laboratorio estuvo en analizar las diferencias operativas y de diseño entre ambos enfoques. Mientras que el método de Lagrange resulta intuitivo y directo para conjuntos de datos estáticos y pequeños, su reestructura completa ante la adición de un nuevo nodo representa una desventaja importante en términos de cómputo. Por otro lado, la estructura matricial y progresiva de las diferencias divididas de

Newton demostró ser una solución mucho más versátil y eficiente cuando se trabaja en entornos dinámicos donde se incorporan mediciones continuas, ya que permite reutilizar el trabajo previo sin necesidad de empezar desde cero.

En conclusión, la práctica cumplió con su objetivo al permitirme comprender no solo cómo programar estos métodos y corregir su lógica interna, sino también cómo tomar decisiones de diseño computacional fundamentadas en la eficiencia algorítmica y en las necesidades reales del problema a resolver.

7. Referencias Bibliográficas (Formato APA)

• Burden, R. L., & Faires, J. D. (2017). Análisis Numérico (10a. ed.). Cengage Learning.

• Chapra, S. C., & Canale, R. P. (2015). Métodos Numéricos para Ingenieros (7a. ed.). McGraw-Hill Education.